

Machine Learning (CE 40717)

Fall 2024

Ali Sharifi-Zarchi

CE Department
Sharif University of Technology

October 14, 2024



1 Introduction

Condorcet's jury theorem

Ensemble learning

Ensemble Methods

2 Bagging

3 Boosting

4 AdaBoost

5 Comparison

6 References

1 Introduction

Condorcet's jury theorem

Ensemble learning

Ensemble Methods

② Bagging

③ Boosting

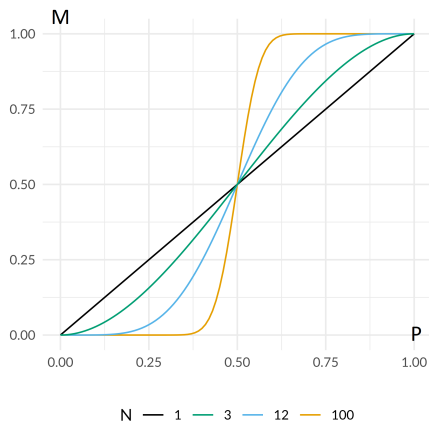
4 AdaBoost

5 Comparison

6 References

Condorcet's jury theorem

- N voters wish to reach a decision by **majority vote**.
- Each voter has an independent probability p of voting for the correct decision.
- Let M be the probability of the majority voting for the correct decision.
- If $p > 0.5$ and $N \rightarrow \infty$, then $M \rightarrow 1$
 - How?



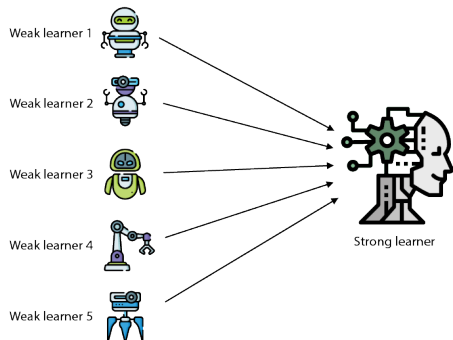
Adopted from Wikipedia

Strong vs. weak Learners

- **Strong learner:** we seek to produce one classifier for which the classification error can be made arbitrarily small.
 - So far we were looking for such methods.
- **Weak learner:** a classifier which is just better than random guessing (for now this will be our only expectation).

Basic idea

- Certain **weak learners** do well in modeling one aspect of the data, while others do well in modeling another.
- Learn several simple models and **combine** their outputs to produce the final decision.
- A **composite prediction** where the final accuracy is **better** than the accuracy of **individual models**.

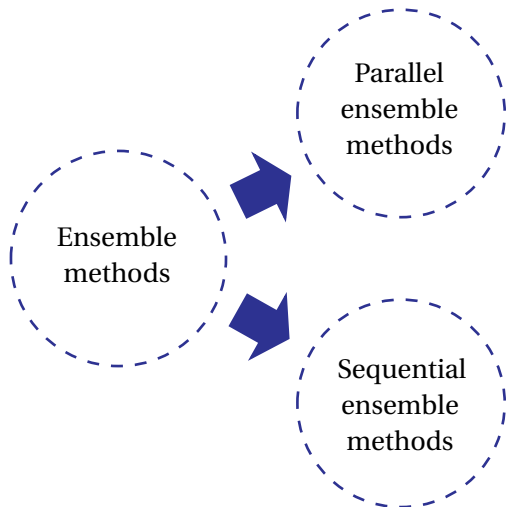


Adopted from [4]

6 References

Ensemble Methods

Ensemble Methods



- Weak learners are generated in **parallel**.
- Basic motivation is to use **independence** between the learners.

- Weak learners are generated **consecutively**.
- Basic motivation is to use **dependence** between the base learners.

What we talk about

- Weak or simple learners
 - **Low variance**: they don't usually overfit
 - **High bias**: they can't learn complex functions
- **Bagging** (parallel): To decrease the variance
 - Random Forest
- **Boosting** (sequential): To decrease the bias (enhance their capabilities)
 - AdaBoost



1 Introduction

2 Bagging

Basic idea & algorithm

Decision tree (quick review)

Random Forest

③ Boosting

4 AdaBoost

5 Comparison

6 References

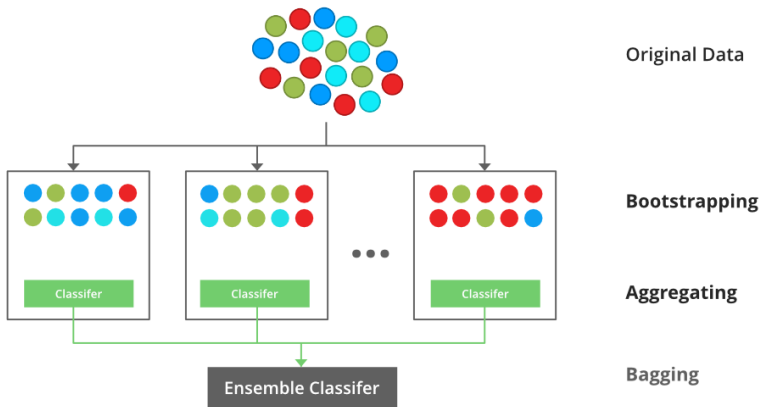
Basic idea



- **Bagging** = **B**ootstrap **a**ggregating
- It uses **bootstrap resampling** to generate different training datasets from the original training dataset.
 - Samples training data uniformly at random with replacement.
- On the training datasets, it trains different weak learners.
- During testing, it **aggregates** the weak learners by uniform averaging or majority voting.
 - Works best with unstable models (high variance models). Why?



Basic idea, Cont.

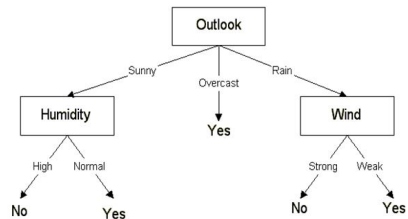


Adopted from GeeksForGeeks

Structure



- **Terminal nodes** (leaves) represent target variable.
- Each **internal node** denotes a test on an attribute.



Outlook	Temperature	Humidity	Wind	Played football(yes/no)
Sunny	Hot	High	Weak	No
Sunny	Hot	High	Strong	No
Overcast	Hot	High	Weak	Yes
Rain	Mild	High	Weak	Yes
Rain	Cool	Normal	Weak	Yes
Rain	Cool	Normal	Strong	No
Overcast	Cool	Normal	Strong	Yes
Sunny	Mild	High	Weak	No
Sunny	Cool	Normal	Weak	Yes
Rain	Mild	Normal	Weak	Yes
Sunny	Mild	Normal	Strong	Yes
Overcast	Mild	High	Strong	Yes
Overcast	Hot	Normal	Weak	Yes
Rain	Mild	High	Strong	No

Adopted from Medium

Algorithm 2 Constructing DT

```

1: procedure FINDTREE( $S, A$ ) ▷ Input:  $S$  (samples),  $A$  (attributes)
2:   if  $A$  is empty or all labels in  $S$  are the same then
3:     status  $\leftarrow$  leaf
4:     class  $\leftarrow$  most common class in  $S$ 
5:   else
6:     status  $\leftarrow$  internal
7:      $a \leftarrow$  bestAttribute( $S, A$ ) ▷ The attribute value test
8:     LeftNode  $\leftarrow$  FindTree( $S(a > t), A - \{a\}$ ) ▷  $t$  (threshold)
9:     RightNode  $\leftarrow$  FindTree( $S(a \leq t), A - \{a\}$ )
10:   end if
11: end procedure

```

Which attribute is the best?

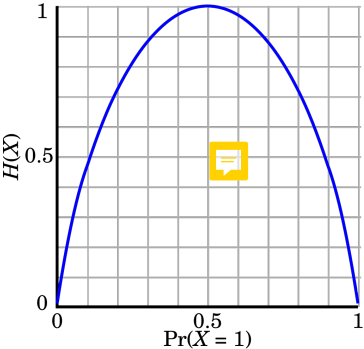
- Entropy** measures the uncertainty in a specific distribution.

$$H(X) = - \sum_{x_i \in \mathcal{X}} P(x_i) \log_2 P(x_i)$$

- Information Gain (IG)**

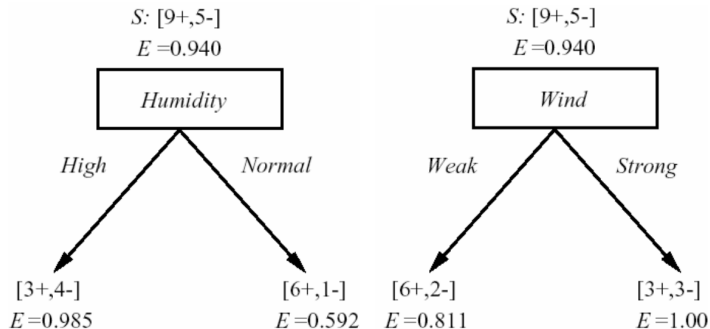
$$\text{Gain}(S, A) = H_S(Y) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} H_{S_v}(Y)$$

A: variable used to split samples
 Y: target variable
 S: samples, S_v : subset of S where $A = v$
 $H_S(Y)$: entropy of Y over S



Adopted from Wikipedia

Example



Adopted from [5]

$$\text{Gain}(S, \text{Humidity}) = 0.940 - (7/14)0.985 - (7/14)0.592 = 0.151$$

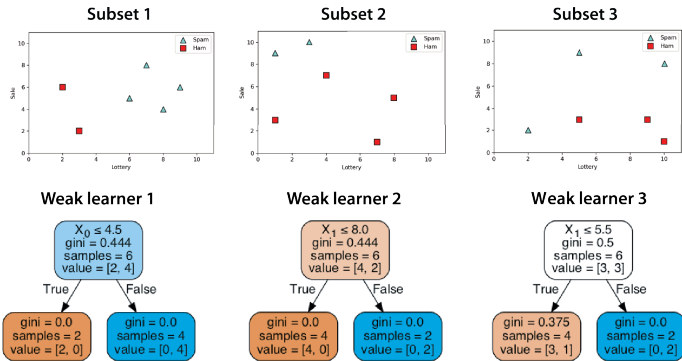
$$\text{Gain}(S, \text{Wind}) = 0.940 - (8/14)0.811 - (6/14)1.0 = 0.48$$



Bagging on decision trees?

Why decision trees?

- Interpretable
- Robust to outliers
- **Low bias**
- **High variance**



Adopted from [4]

Perfect candidates

- Why are **DTs** good candidates for ensembles?
 - Consider averaging many (nearly) **unbiased** tree estimators.
 - Bias remains similar, but **variance is reduced**.
- Remember Bagging?
 - Train many trees on bootstrapped data, then aggregate (average/majority) the outputs.

Algorithm



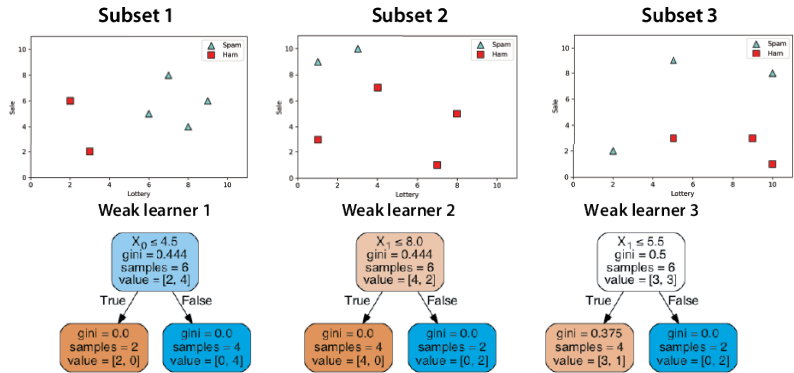
Algorithm 3 Random Forest

- 1: **Input:** T (number of trees), m (number of variables used to split each node)
- 2: **for** $t = 1$ to T **do**
- 3: Draw a bootstrap dataset
- 4: **Select** m features randomly out of d features as candidates for splitting
- 5: Learn a tree on this dataset
- 6: **end for**
- 7: **Output:**
- 8: Regression: average of the outputs
- 9: Classification: majority voting

▷ Usually: $m = \sqrt{d}$

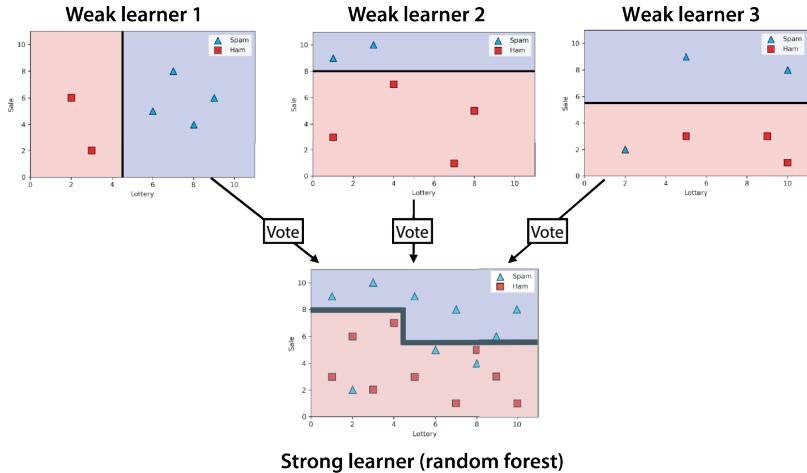


Example, Cont.



Adopted from [4]

Example, Cont.



Adopted from [4]

1 Introduction

2 Bagging

3 Boosting
Motivation & basic idea
Algorithm

4 AdaBoost

5 Comparison

6 References

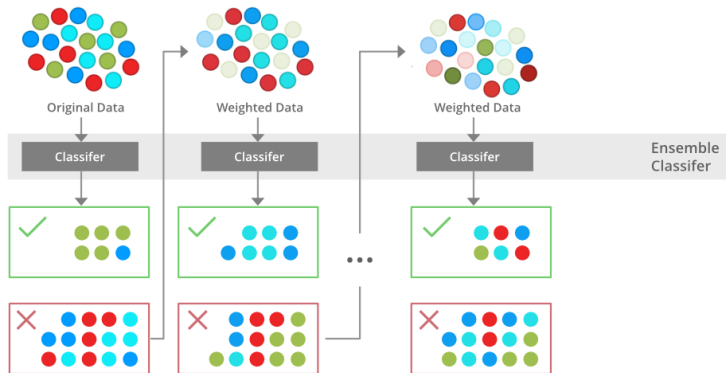
Problems with bagging

- Bagging created a diversity of **weak learners** by creating random datasets.
 - Examples: Decision stumps (shallow decision trees), Logistic regression, ...
- Did we have full control over the usefulness of the weak learners?
 - The **diversity** or **complementarity** of the weak learners is not controlled in any way, it is left to chance and to the instability (variance) of the models.

Basic idea

- We would expect a better performance if the weak learners also **complemented** each other.
 - They would have "expertise" on different subsets of the dataset.
 - So they would work better on different subsets.
- The basic idea of boosting is to generate a **series** of weak learners which complement each other.
 - For this, we will force each learner to focus on **the mistakes of the previous learner**.

Basic idea, Cont.



Adopted from GeeksForGeeks

1 Introduction

2 Bagging

3 Boosting

Motivation & basic idea

Algorithm

4 AdaBoost

5 Comparison

6 References

Algorithm

- Try to combine many simple **weak** learners (in sequence) to find a single **strong** learner (For simplicity, suppose that we have a classification problem from now on).
 - Each component is a simple binary ± 1 classifier
 - Voted combinations of component classifiers

$$H_M(\mathbf{x}) = \alpha_1 h(\mathbf{x}; \boldsymbol{\theta}_1) + \cdots + \alpha_M h(\mathbf{x}; \boldsymbol{\theta}_M)$$



- To simplify notations: $h(\mathbf{x}; \boldsymbol{\theta}_i) = h_i(\mathbf{x})$

$$H_M(\mathbf{x}) = \alpha_1 h_1(\mathbf{x}) + \cdots + \alpha_M h_M(\mathbf{x})$$

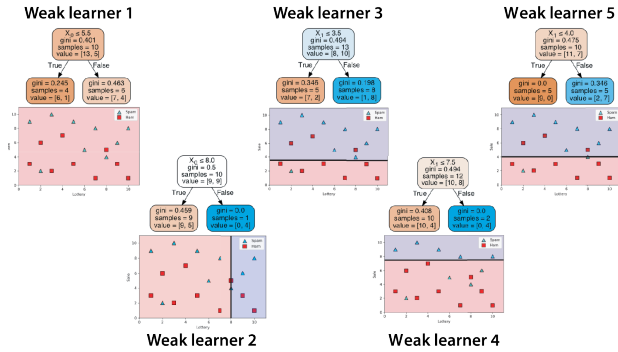
- **Prediction:** $\hat{y} = \text{sign}(H_M(\mathbf{x}))$

Candidate for $h_i(x)$

- Decision stumps
- Each classifier is based on only a single feature of $\mathbf{x}$ (e.g., $\mathbf{x}_k$):

$$h(\mathbf{x};\boldsymbol{\theta}) = \text{sign}(w_1\mathbf{x}_k - w_0)$$

$$\boldsymbol{\theta} = \{k, w_1, w_0\}$$



Adopted from [4]

1 Introduction

2 Bagging

3 Boosting

4 AdaBoost

Basic idea & algorithm

Loss function & proof

Properties (extra-reading)

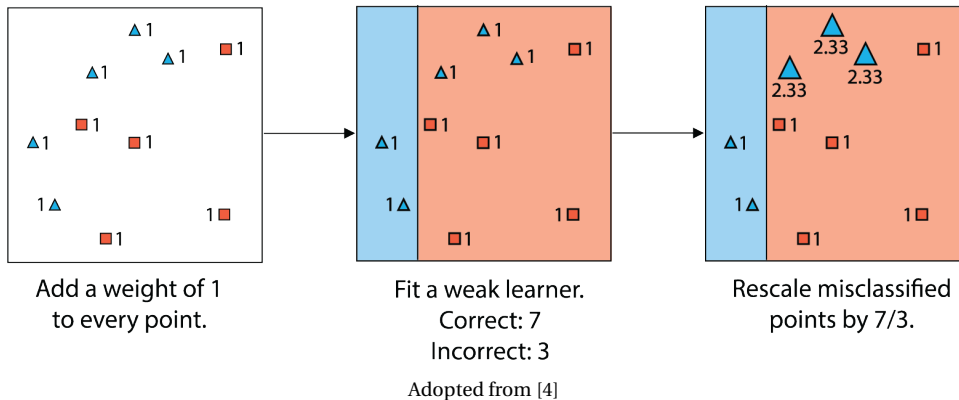
5 Comparison

6 References

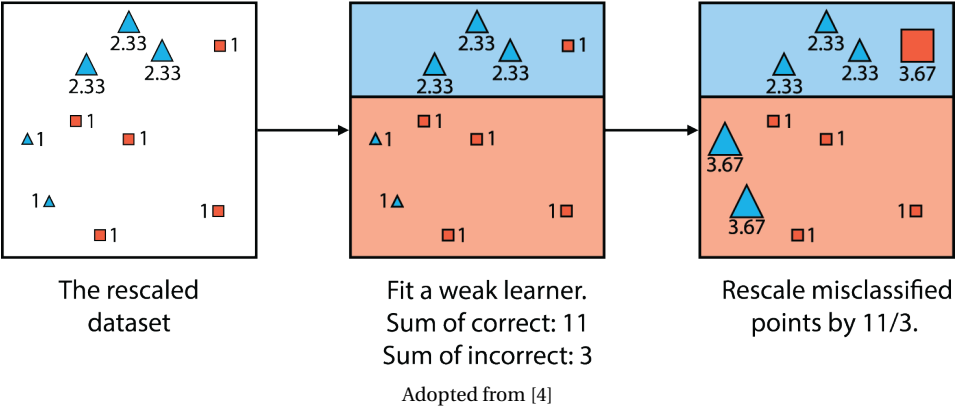
Basic idea

- **Sequential** production of classifiers
 - Iteratively add the classifier whose addition will be most helpful.
- Represent the importance of each sample by assigning **weights** to them.
 - Correct classification $\Rightarrow$ smaller weights
 - Misclassified samples $\Rightarrow$ larger weights
- Each classifier is **dependent** on the previous ones.
 - Focuses on the **previous ones' error**.

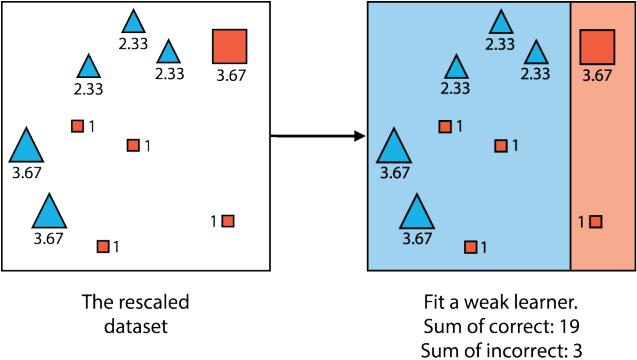
Example



Example, Cont.

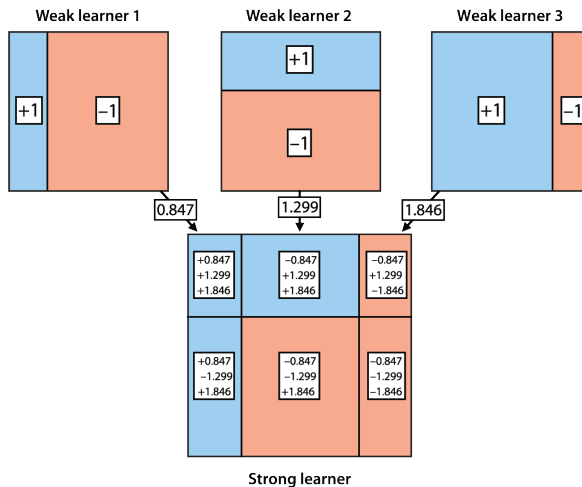


Example, Cont.



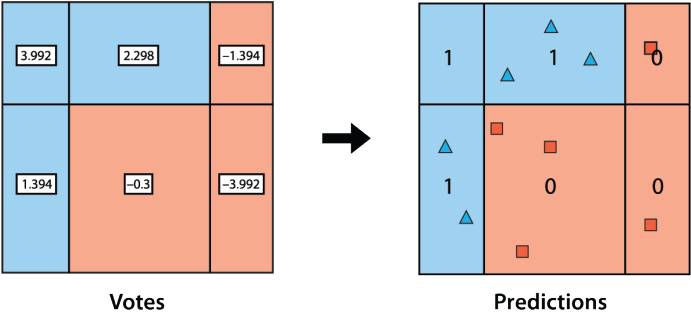
Adopted from [4]

Example, Cont.



Adopted from [4]

Example, Cont.



Adopted from [4]

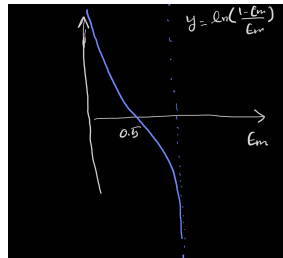
Algorithm



- $H_M(\mathbf{x}) = \frac{1}{2}[\alpha_1 h_1(\mathbf{x}) + \dots + \alpha_M h_M(\mathbf{x})] \longrightarrow$ the complete model $y^{(i)} \in \{-1, 1\}$
 - $h_m(\mathbf{x})$: m -th weak learner
 - $\alpha_m = ? \longrightarrow$ votes of the m -th weak learner
- $w_m^{(i)}$: weight of sample i in iteration m
 - $w_{m+1}^{(i)} = ?$
- $J_m = \sum_{i=1}^N w_m^{(i)} \times I(y^{(i)} \neq h_m(\mathbf{x}^{(i)})) \longrightarrow$ loss of the m -th weak learner
- $\epsilon_m = \frac{\sum_{i=1}^N w_m^{(i)} \times I(y^{(i)} \neq h_m(\mathbf{x}^{(i)}))}{\sum_{i=1}^N w_m^{(i)}} \longrightarrow$ weighted error of the m -th weak learner

Algorithm, Cont.

- $H_M(\mathbf{x}) = \frac{1}{2}[\alpha_1 h_1(\mathbf{x}) + \dots + \alpha_M h_M(\mathbf{x})] \rightarrow$ the complete model $y^{(i)} \in \{-1, 1\}$
 - $h_m(\mathbf{x})$: m -th weak learner
 - $\alpha_m = \ln\left(\frac{1 - \epsilon_m}{\epsilon_m}\right) \rightarrow$ votes of the m -th weak learner
- $w_m^{(i)}$: weight of sample i in iteration m
 - $w_{m+1}^{(i)} = w_m^{(i)} e^{\alpha_m I(y^{(i)} \neq h_m(\mathbf{x}^{(i)}))}$
- $J_m = \sum_{i=1}^N w_m^{(i)} \times I(y^{(i)} \neq h_m(\mathbf{x}^{(i)})) \rightarrow$ loss of the m -th weak learner
- $\epsilon_m = \frac{\sum_{i=1}^N w_m^{(i)} \times I(y^{(i)} \neq h_m(\mathbf{x}^{(i)}))}{\sum_{i=1}^N w_m^{(i)}} \rightarrow$ weighted error of the m -th weak learner



Algorithm, Cont.

Algorithm 4 AdaBoost

1: Initialize data weight $w_1^{(i)} = \frac{1}{N}$ for all N samples

2: **for** $m = 1$ to M **do**

3: Find $h_m(\mathbf{x})$ by minimizing the loss:

$$J_m = \sum_{i=1}^N w_m^{(i)} \times I(y^{(i)} \neq h_m(\mathbf{x}^{(i)}))$$

4: Find the weighted error of $h_m(\mathbf{x})$:

$$\epsilon_m = \frac{\sum_{i=1}^N w_m^{(i)} \times I(y^{(i)} \neq h_m(\mathbf{x}^{(i)}))}{\sum_{i=1}^N w_m^{(i)}}$$

5: Assign votes $\alpha_m = \ln\left(\frac{1 - \epsilon_m}{\epsilon_m}\right)$

6: Update the weights:

$$w_{m+1}^{(i)} = w_m^{(i)} e^{\alpha_m I(y^{(i)} \neq h_m(\mathbf{x}^{(i)}))}$$

7: **end for**

8: **Combined classifier:** $\hat{y} = \text{sign}(H_M(\mathbf{x}))$ where $H_M(\mathbf{x}) = \frac{1}{2} \sum_{m=1}^M \alpha_m h_m(\mathbf{x})$

1 Introduction

2 Bagging

3 Boosting

4 AdaBoost

Basic idea & algorithm

Loss function & proof

Properties (extra-reading)

5 Comparison

6 References

Loss function

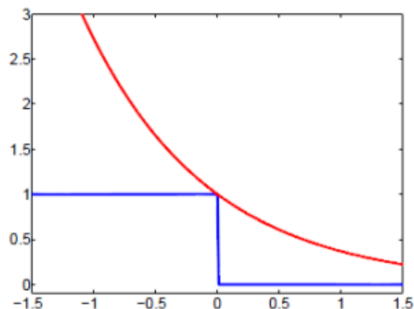
- There are many options for the loss function.
 - AdaBoost is equivalent to using the following **exponential loss**.

$$\mathcal{L}(y, H_M(\mathbf{x})) = e^{-y \times H_M(\mathbf{x})}$$

$$\hat{y} = \text{sign}(H_M(\mathbf{x}))$$

Why the exponential loss?

- Differentiable approximation (bound) of the **0/1 loss**
 - Easy to optimize
 - Optimizing an upper bound on classification error.



 $^{(i)}H_M(x^{(i)})$

Adopted from [2]

Step 1: Calculating the exponential loss

- We need to calculate the exponential loss for:

$$H_m(\mathbf{x}) = \frac{1}{2} [\alpha_1 h_1(\mathbf{x}) + \dots + \alpha_m h_m(\mathbf{x})]$$

To have a cleaner form later

- **Idea:** consider adding the m -th component:

$$\begin{aligned}\mathcal{L}_m &= \sum_{i=1}^N e^{-y^{(i)} H_m(\mathbf{x}^{(i)})} = \sum_{i=1}^N e^{-y^{(i)} [H_{m-1}(\mathbf{x}^{(i)}) + \frac{1}{2} \alpha_m h_m(\mathbf{x}^{(i)})]} \\ &= \sum_{i=1}^N e^{-y^{(i)} H_{m-1}(\mathbf{x}^{(i)})} e^{-\frac{1}{2} \alpha_m y^{(i)} h_m(\mathbf{x}^{(i)})} = \sum_{i=1}^N \underbrace{w_m^{(i)}}_{e^{-y^{(i)} H_{m-1}(\mathbf{x}^{(i)})}} e^{-\frac{1}{2} \alpha_m y^{(i)} h_m(\mathbf{x}^{(i)})}\end{aligned}$$

Suppose it is fixed at stage m

Should be optimized at stage m by seeking $h_m(\mathbf{x})$ and α_m

Step 2: Deriving the weighted error function

- We need to derive the weighted error function, J_m

$$\begin{aligned}
 \mathcal{L}_m &= \sum_{i=1}^N w_m^{(i)} e^{-\frac{1}{2} \alpha_m y^{(i)} h_m(\mathbf{x}^{(i)})} \\
 &= e^{\frac{-\alpha_m}{2}} \left(\sum_{y^{(i)} = h_m(\mathbf{x}^{(i)})} w_m^{(i)} \right) + e^{\frac{\alpha_m}{2}} \left(\sum_{y^{(i)} \neq h_m(\mathbf{x}^{(i)})} w_m^{(i)} \right) \\
 &= (e^{\frac{\alpha_m}{2}} - e^{\frac{-\alpha_m}{2}}) \underbrace{\left(\sum_{y^{(i)} \neq h_m(\mathbf{x}^{(i)})} w_m^{(i)} \right)}_{J_m} + e^{\frac{-\alpha_m}{2}} \left(\sum_{i=1}^N w_m^{(i)} \right) \\
 J_m &= \sum_{i=1}^N w_m^{(i)} \times I \left(y^{(i)} \neq h_m(\mathbf{x}^{(i)}) \right)
 \end{aligned}$$

Find $h_m(\mathbf{x})$ that minimizes J_m

Step 3: Deriving ϵ_m and α_m

- We need to derive ϵ_m and α_m by setting the derivative equal to zero:

$$\frac{\partial \mathcal{L}_m}{\partial \alpha_m} = 0$$

- Idea:** separate the derivative into misclassified and correctly classified samples.

$$\Rightarrow \frac{1}{2} (e^{\frac{\alpha_m}{2}} + e^{-\frac{\alpha_m}{2}}) \left(\sum_{y^{(i)} \neq h_m(\mathbf{x}^{(i)})} w_m^{(i)} \right) = \frac{1}{2} e^{-\frac{\alpha_m}{2}} \left(\sum_{i=1}^N w_m^{(i)} \right)$$

$$\Rightarrow \frac{e^{-\frac{\alpha_m}{2}}}{(e^{\frac{\alpha_m}{2}} + e^{-\frac{\alpha_m}{2}})} = \frac{\sum_{y^{(i)} \neq h_m(\mathbf{x}^{(i)})} w_m^{(i)}}{\sum_{i=1}^N w_m^{(i)}}$$

- Set $\epsilon_m = \frac{\sum_{i=1}^N w_m^{(i)} I(y^{(i)} \neq h_m(\mathbf{x}^{(i)}))}{\sum_{i=1}^N w_m^{(i)}} \Rightarrow \alpha_m = \ln \left(\frac{1 - \epsilon_m}{\epsilon_m} \right)$

Step 4: Justifying the weight update mechanism

- We need to justify the weight update mechanism.
- Idea:** we have $w_m^{(i)}$ from the first step as $w_{m+1}^{(i)} = e^{-y^{(i)} H_m(\mathbf{x}^{(i)})}$

$$\xRightarrow{\text{separate } h_m(\mathbf{x}^{(i)})} w_{m+1}^{(i)} = w_m^{(i)} e^{-\frac{1}{2} \alpha_m y^{(i)} h_m(\mathbf{x}^{(i)})}$$

$$\xRightarrow{y^{(i)} h_m(\mathbf{x}^{(i)}) = 1 - 2I(y^{(i)} \neq h_m(\mathbf{x}^{(i)}))} w_{m+1}^{(i)} = w_m^{(i)} e^{-\frac{1}{2} \alpha_m} e^{\alpha_m I(y^{(i)} \neq h_m(\mathbf{x}^{(i)}))}$$

↑

Independent of i and can be ignored

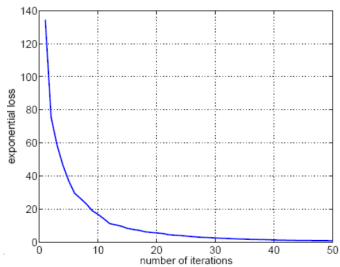
$$\Rightarrow w_{m+1}^{(i)} = w_m^{(i)} e^{\alpha_m I(y^{(i)} \neq h_m(\mathbf{x}^{(i)}))}$$

Exponential loss properties

- In each boosting iteration, assuming we can find $h(\mathbf{x}; \boldsymbol{\theta}_m)$ whose weighted error is better than chance.

$$H_m(x) = \frac{1}{2} [\alpha_1 h(\mathbf{x}; \boldsymbol{\theta}_1) + \cdots + \alpha_m h(\mathbf{x}; \boldsymbol{\theta}_m)]$$

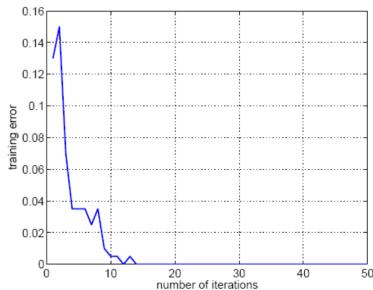
- Thus, **lower exponential loss** over training data is guaranteed.



Adopted from [6]

Training error properties

- Boosting iterations typically **decrease** the **training error** of $H_M(\mathbf{x})$ over training examples.

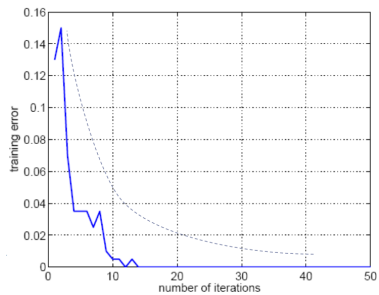


Adopted from [6]

Training error properties, Cont.

- Training error** has to go **down exponentially fast** if the weighted error of each h_m is strictly better than chance (i.e., $\epsilon_m < 0.5$)

$$E_{\text{train}}(H_M) \leq \prod_{m=1}^M 2\sqrt{\epsilon_m(1-\epsilon_m)}$$

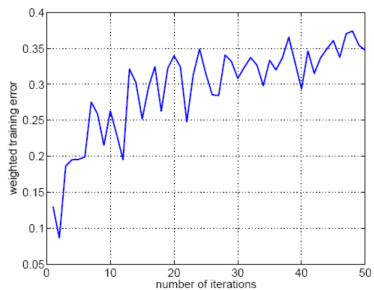


Adopted from [6]

Weighted error properties

- Weighted error** of each new component classifier tends to **increase** as a function of boosting iterations.

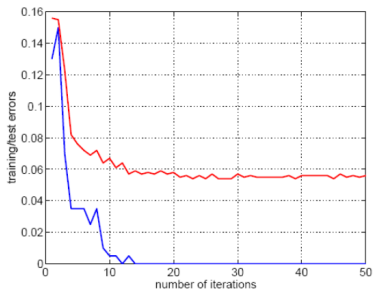
$$\epsilon_m = \frac{\sum_{i=1}^N w_m^{(i)} I(y^{(i)} \neq h_m(\mathbf{x}^{(i)}))}{\sum_{i=1}^N w_m^{(i)}}$$



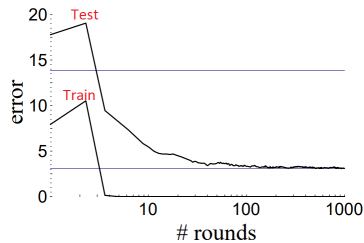
Adopted from [6]

Test error properties

- **Test error** can still **decrease** after training error is flat (even zero).
- But, is it robust to overfitting?
 - May easily overfit in the presence of labeling noise or overlap of classes.



Adopted from [6]



Adopted from [3]

Typical behavior

- **Exponential loss goes strictly down.**
- **Training error of H goes down.**
- Weighted error ϵ_m goes **up** $\implies$ share of votes α_m goes **down**.
- **Test error decreases** even after a flat training error.

Bagging vs. Boosting

	Bagging	Boosting
Training Strategy	Parallel training	Sequential training
Data Sampling	Bootstrapping (random subsets)	Weighted (by instance importance)
Learners Dependency	Independent	Dependent (on the previous models)
Learner Weighting	Equal weights	Varying weights (based on importance)
Tolerance to Noise	More robust (due to aggregation)	More sensitive (may overfit to noise)
Properties	Reduces variance	Reduces bias and variance (focus on bias)

- 1 Introduction
- 2 Bagging
- 3 Boosting
- 4 AdaBoost
- 5 Comparison
- 6 References

Contributions

- **This slide has been prepared thanks to:**
 - Nikan Vasei
 - Mahan Bayhaghi

- [1] C. M., *Pattern Recognition and Machine Learning*.
Information Science and Statistics, New York, NY: Springer, 1 ed., Aug. 2006.
- [2] M. Soleymani Baghshah, “Machine learning.” Lecture slides.
- [3] R. E. Schapire, “The boosting approach to machine learning: An overview,”
Nonlinear estimation and classification, pp. 149–171, 2003.
- [4] L. Serrano, *Grokking machine learning*.
New York, NY: Manning Publications, Jan. 2022.
- [5] T. Mitchell, *Machine Learning*.
McGraw-Hill series in computer science, New York, NY: McGraw-Hill Professional,
Mar. 1997.
- [6] T. Jaakkola, “Machine learning course slides.” Lecture slides.